

TECHNICAL DOCUMENTATION

AI-Proctored Multiple-Choice Quiz Application

Architecture, Attention-Monitoring Engine, and API Reference

DOCUMENT TYPE	System & Code Documentation
FRAMEWORK	FastAPI + Jinja2 + OpenAI Agents SDK
PROCTORING ENGINE	OpenCV + MediaPipe Face Landmarker
VERSION	1.0
PREPARED	July 2026

Table of Contents

1. Overview

- Purpose
- High-Level Architecture

2. Quiz Generation Flow

- Data Schemas
- Concept & Question Agents
- Retry Handling

3. Application Routes (API Reference)

4. Attention Monitoring Engine

- Configuration Parameters
- Calibration Phase
- Detection Signals
- Warning & Disqualification Logic
- Video Feed Streaming

5. Front-End Behavior (Question Page)

6. Templates Overview

7. Session State Reference

8. Configuration & Environment Variables

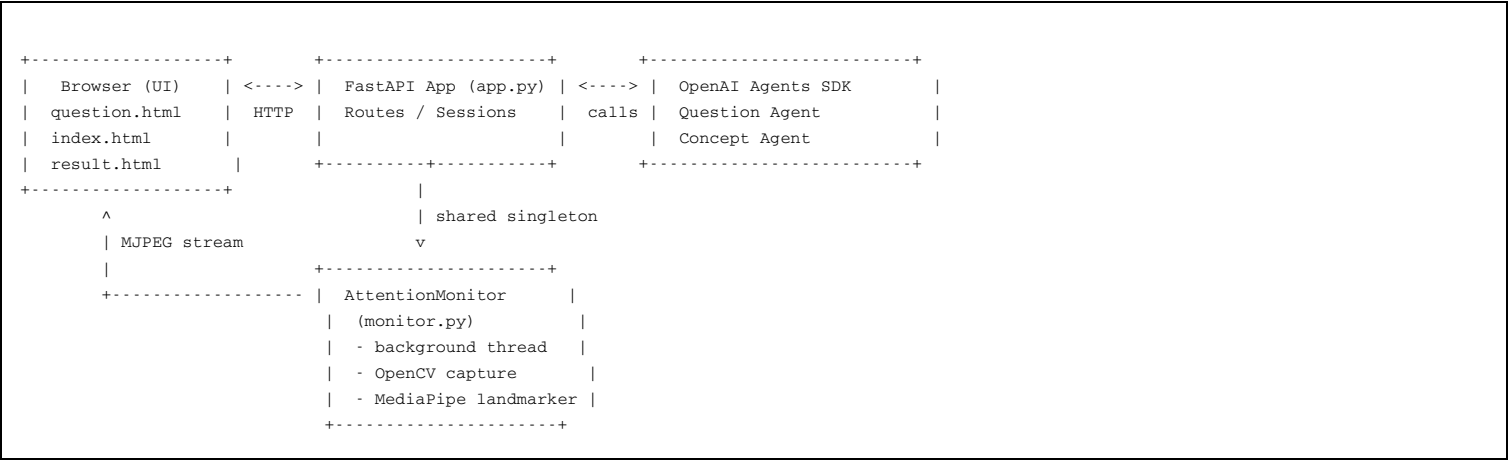
9. Known Limitations & Notes

1. Overview

1.1 Purpose

This application delivers a topic-based, AI-generated multiple-choice quiz while monitoring the test-taker's webcam feed for signs of distraction or dishonest behavior. Questions are generated on demand using two LLM-backed agents, served one at a time through a FastAPI web server, and each question is subject to a countdown timer. In parallel, a background thread analyzes webcam frames using MediaPipe's Face Landmarker to detect gaze deviation, head-turning, closed eyes, absence of a face, or multiple faces in frame. Detected anomalies accumulate as "warnings"; once a configurable warning limit is reached, the quiz is automatically terminated and the candidate is routed to a "disqualified" result page.

1.2 High-Level Architecture



KEY DESIGN POINT

The `AttentionMonitor` is instantiated once as a module-level singleton (`monitor`) and shared across all requests. It runs its capture-and-analysis loop on a dedicated background thread so that the FastAPI event loop is never blocked by webcam I/O or model inference.

2. Quiz Generation Flow

2.1 Data Schemas

Two Pydantic models constrain the structured output returned by the LLM agents:

MODEL	FIELD	DESCRIPTION
QuestionStructure	Question	A clear, grammatically correct MCQ prompt
	Options	Exactly four unique answer choices
	Answer	The exact text of the correct option
ConceptList	Concepts	A list of distinct sub-concepts for a topic

2.2 Concept & Question Agents

Quiz generation is a two-stage pipeline. First, the **Concept Agent** expands a topic string (e.g. "Photosynthesis") into a list of distinct sub-concepts — one per requested question. Then, for each sub-concept, the **Question Agent** produces one four-option multiple-choice question at the requested difficulty.

```
Question_agent = Agent(
    name="Question Agent",
    instructions="Generate one high-quality MCQ with exactly four options ...",
    output_type=QuestionStructure,
)

Concept_agent = Agent(
    name="Concept Agent",
    instructions="Generate the exact number of distinct concepts requested ...",
    output_type=ConceptList,
)
```

If the Concept Agent returns fewer concepts than requested, `generate_concepts()` pads the list by cycling through existing entries rather than failing the request outright; if it returns more, the list is truncated to the requested count.

2.3 Retry Handling

Both agent calls are wrapped by `_run_with_retries()`, which retries a failed `Runner.run()` call up to `AGENT_MAX_RETRIES` times (default 2), with an increasing back-off delay of `AGENT_RETRY_DELAY_SECONDS × attempt` seconds between tries. If all attempts fail, the original exception propagates and the `/generate` route surfaces a user-facing error on the topic-selection page.

3. Application Routes (API Reference)

ROUTE	METHOD	DESCRIPTION
/	GET	Renders the topic-selection page. Stops the monitor (in case a prior session was left running) and surfaces any queued session error.
/generate	POST	Accepts <code>topic</code> , <code>difficulty</code> , and <code>num_questions</code> . Validates inputs against allowed value sets, generates concepts and questions via the agents, resets session state (score, answers, warnings), starts the <code>AttentionMonitor</code> , and redirects to question 0.
/question/{idx}	GET	Renders a single question. Redirects to <code>/result</code> if the monitor has already flagged disqualification, or if <code>idx</code> is out of range. Records <code>question_started_at</code> in the session.
/answer/{idx}	POST	Grades the submitted <code>option</code> against the stored correct answer, appends a review record, checks disqualification status both before and after grading, then redirects to the next question or to <code>/result</code> .
/result	GET	Renders the final score, per-question review trail, and warning history. Stops the monitor to release the webcam.
/restart	GET	Stops the monitor, clears the session entirely, and redirects to the topic-selection page.
/api/monitor-status	GET	Returns the monitor's current JSON status. Polled every 2 seconds by <code>question.html</code> so a mid-question warning-limit breach ends the quiz immediately without waiting for the next form submission.
/api/report-violation	POST	Receives client-detected violations (tab switch, window blur) as JSON <code>{"reason": "..."}"</code> and forwards them to <code>monitor.report_external_violation()</code> .
/video-feed	GET	Streams the annotated webcam preview as multipart MJPEG (<code>multipart/x-mixed-replace</code>) for display in the browser.

4. Attention Monitoring Engine

The proctoring logic lives in `monitor.py`, inside the `AttentionMonitor` class. It owns a background thread that continuously reads frames from the local webcam (device index 0), runs MediaPipe's `FaceLandmarker` task, derives gaze and head-pose metrics, and converts sustained anomalies into warnings.

4.1 Configuration Parameters

CONSTANT	DEFAULT	MEANING
<code>WARNING_LIMIT</code>	5	Warnings allowed before disqualification
<code>DEFAULT_GAZE_H_RANGE</code>	0.42 – 0.58	Baseline horizontal iris-position window
<code>DEFAULT_GAZE_V_RANGE</code>	0.40 – 0.60	Baseline vertical iris-position window
<code>DEFAULT_HEAD_H_RANGE</code>	0.45 – 0.55	Baseline horizontal head-yaw window
<code>DEFAULT_HEAD_V_RANGE</code>	0.40 – 0.60	Baseline vertical head-pitch window
<code>EYE_CLOSED_RATIO</code>	0.18	Eye-aspect-ratio threshold below which eyes are considered closed
<code>EYE_CLOSED_DURATION_SECONDS</code>	1.2 s	Minimum closed-eye duration before flagging
<code>WARNING_INTERVAL_SECONDS</code>	4 s	Base cadence at which a sustained violation converts to a warning
<code>GRACE_PERIOD_SECONDS</code>	3 s	Initial window used purely for calibration; no warnings issued
<code>CALIBRATION_MIN_SAMPLES</code>	15	Minimum frames required to trust a calibrated baseline
<code>FACE_COUNT_CONFIRM_FRAMES</code>	6	Consecutive frames required before "no face" / "multiple faces" is flagged
<code>REASON_VOTE_WINDOW</code> / <code>REASON_VOTE_RATIO</code>	10 / 0.6	Rolling majority vote used to smooth noisy gaze/head signals
<code>WEBCAM_OPEN_RETRIES</code>	3	Attempts to open the webcam device before giving up

4.2 Calibration Phase

During the first `GRACE_PERIOD_SECONDS` of a session, no warnings are issued. Instead, gaze and head-pose samples are collected. When the grace period ends, `_finish_calibration()` computes the median of each metric and re-centers the acceptable ranges around it (using tolerance constants such as `CALIBRATION_TOLERANCE_GAZE`), producing a per-candidate baseline. If fewer than `CALIBRATION_MIN_SAMPLES` were collected, the engine falls back to the hard-coded default ranges.

4.3 Detection Signals

SIGNAL	METHOD
Gaze deviation	Normalized horizontal/vertical iris position relative to eye-corner landmarks (points 468/473 vs. 33/133/362/263, etc.), smoothed over an 8-frame buffer
Head pose (yaw/pitch)	Nose-tip landmark position relative to left/right cheek and top/bottom face-boundary landmarks
Eyes closed	Eye-aspect-ratio (EAR) computed from vertical/horizontal eyelid landmark distances, sustained beyond <code>EYE_CLOSED_DURATION_SECONDS</code>
No face detected	Zero faces returned by the landmarker for <code>FACE_COUNT_CONFIRM_FRAMES</code> consecutive frames
Multiple faces detected	More than one face returned, sustained over the same confirm-frame window
Tab / window focus loss	Reported by the browser via <code>/api/report-violation</code> , not derived from video

NOISE REDUCTION
Gaze deviation, head-turned-away, and looking-down are each passed through a rolling majority vote (<code>_vote()</code>) requiring at least 60% of the last 10 samples to agree before the reason is treated as active. This avoids single-frame jitter from triggering a warning.

4.4 Warning & Disqualification Logic

Once one or more reasons are active outside the grace period, a violation timer starts. The effective interval before the next warning is the base `WARNING_INTERVAL_SECONDS` multiplied by the smallest applicable factor in `REASON_INTERVAL_MULTIPLIERS` — for example, "Multiple faces detected" escalates almost three times faster ($\times 0.35$) than a plain gaze deviation ($\times 1.0$), while "Looking down" is more forgiving ($\times 1.4$). Each time the elapsed violation time crosses another interval boundary, one warning is added, a history entry (reason list, timestamp, and JPEG snapshot) is recorded, and the count is compared against `WARNING_LIMIT`. Reaching the limit sets `disqualified =`

`True`, which is polled by the front end and checked by the `/question`, `/answer`, and `/api/monitor-status` routes.

Non-visual violations (browser tab switches, window blur) bypass the video pipeline entirely and call `report_external_violation()` directly, which adds exactly one warning per reported event.

4.5 Video Feed Streaming

Every processed frame is annotated with a semi-transparent heads-up display showing the current warning count, face/gaze state, and status text (green "FOCUSED", orange "CALIBRATING...", or red with the active reason list and countdown to the next warning). The frame is JPEG-encoded and exposed through `get_frame()`; the FastAPI `/video-feed` route wraps this in an MJPEG generator so the browser can display it as a live `<img>` stream.

5. Front-End Behavior (Question Page)

`question.html` combines four independent client-side mechanisms:

1. **Status polling** — every 2 seconds, fetches `/api/monitor-status` and updates the warning badge; redirects to `/result` immediately if disqualified.
2. **Per-question countdown** — a client-side timer of `question_seconds` (default 45s, from `QUESTION_SECONDS`) that visually drains a progress bar and auto-submits the form (with a blank answer if nothing was selected) when it reaches zero.
3. **Violation reporting** — listens for `visibilitychange` and window `blur` events and POSTs them to `/api/report-violation`, with a 5-second cooldown so a single tab switch is not double-counted. An `isNavigatingAway` flag suppresses false positives caused by the page's own form submission or auto-submit.
4. **Camera feed resilience** — if the `<img>` tag's stream errors out, it is hidden in favor of a "Stream Paused" placeholder and automatically retried after 4 seconds by re-requesting `/video-feed` with a cache-busting query parameter.

6. Templates Overview

TEMPLATE	ROLE
base.html	Shared page shell: header, static stylesheet link, and a content block extended by every other page.
index.html	Topic-selection form: free-text topic, difficulty radio group (easy / medium / hard), and question-count radio group (3 / 5 / 7). Displays a session-stored error message if generation previously failed.
question.html	Single-question view: progress bar, per-question timer bar, optional webcam-unavailable notice, the MCQ form, the live camera preview widget, and all client-side monitoring scripts described in Section 5.
result.html	Final score view. Displays a "Cheating — Quiz Ended" banner when disqualified, otherwise a normal completion message; includes a per-question review trail (selected vs. correct answer) and a full warning log with timestamps and reasons.

7. Session State Reference

The following keys are stored in the signed session cookie (via `SessionMiddleware`) over the course of a quiz attempt:

KEY	TYPE	SET BY	PURPOSE
<code>topic</code>	str	<code>/generate</code>	The user-entered quiz topic
<code>difficulty</code>	str	<code>/generate</code>	easy / medium / hard
<code>questions</code>	list[dict]	<code>/generate</code>	Generated question/option/answer sets
<code>current</code>	int	<code>/generate</code> , <code>/question/{idx}</code>	Index of the question currently displayed
<code>score</code>	int	<code>/generate</code> , <code>/answer/{idx}</code>	Running count of correct answers
<code>warnings_exceeded</code>	bool	multiple routes	Set once the monitor reports disqualification
<code>answers</code>	list[dict]	<code>/answer/{idx}</code>	Per-question review trail (question, selected, correct, is_correct)
<code>question_started_at</code>	float	<code>/question/{idx}</code>	Server-side timestamp when a question was displayed
<code>error</code>	str	<code>/generate</code>	One-shot error message popped and shown on <code>/</code>

8. Configuration & Environment Variables

VARIABLE	DEFAULT	PURPOSE
SECRET_KEY	dev-secret-key-change-me	Session cookie signing key — must be overridden in production
QUESTION_SECONDS	45	Per-question countdown duration
AGENT_MAX_RETRIES	2	Retries for agent calls before failing quiz generation
AGENT_RETRY_DELAY_SECONDS	1.5	Base back-off delay between agent retries
FACE_LANDMARKER_MODEL_PATH	face_landmarker.task next to monitor.py	Path to the MediaPipe face-landmarker model file
WARNING_LIMIT	5	Warnings allowed before disqualification
EYE_CLOSED_RATIO , EYE_CLOSED_DURATION_SECONDS	0.18, 1.2	Drowsiness / eyes-closed detection thresholds
WARNING_INTERVAL_SECONDS	4	Base cadence for converting sustained violations into warnings
GRACE_PERIOD_SECONDS	3	Calibration-only window at session start
CALIBRATION_TOLERANCE_GAZE , _HEAD_H , _HEAD_V	0.08, 0.05, 0.10	Width of the acceptable range around each candidate's calibrated baseline
FACE_COUNT_CONFIRM_FRAMES	6	Debounce for face-count-based flags
REASON_VOTE_WINDOW , REASON_VOTE_RATIO	10, 0.6	Rolling majority-vote smoothing for gaze/head reasons
WEBCAM_OPEN_RETRIES	3	Webcam device-open retry count on startup

9. Known Limitations & Notes

- The monitor opens local webcam device index `0` only; multi-camera or remote-camera setups are not supported without modification.
- Calibration assumes the candidate is looking at the screen in a neutral position during the first `GRACE_PERIOD_SECONDS`; a poor calibration window can shift the effective detection thresholds for the rest of the session.
- Session state — including generated questions and the answer key — is stored in a signed cookie. The default `SECRET_KEY` is not safe for production and must be replaced with a securely generated secret.
- Warning snapshots are embedded as base64 JPEG strings directly inside the session-independent monitor history, held in server memory rather than persisted to disk or a database.
- The `AttentionMonitor` is a single process-wide singleton; concurrent quiz sessions from multiple browser tabs or users will share one webcam and one warning count.

— End of Document —